

Lieferpaket für Demos & Games — Amiga-Core (MEGA65)

Juni 2026

Lieferpaket für Demos & Games — Anleitung für deft

1. Worum es geht

Der MEGA65-Amiga-Core bekommt einen **Loader**: Er lädt ein Speicherabbild ("RamDump") von der SD-Karte direkt ins Chip- und Slow-RAM und springt dann kalt an eine definierte Entry-Adresse. Es gibt dabei **kein Kickstart** zur Laufzeit (im ROM-Bereich liegt unser Mini-Launcher), **kein exec, kein dos, kein track-disk, kein Floppy-Laufwerk** — die Demo bekommt die nackte, frisch resettete Maschine plus fertig befüllten Speicher.

Du lieferst Rohmaterial (Speicherinhalte + eine kurze formlose Notiz, siehe Abschnitt 5). Das Paketieren ins eigentliche Dateiformat übernehmen wir — du brauchst kein Tool von uns.

2. Die Ziel-Maschine

Eigenschaft	Wert
CPU	68000 (zyklusgenau, fx68k)
Chipsatz	OCS (A500), PAL — keine ECS-/AGA-Register
Chip RAM	512 KB, \$000000–\$07FFFF
Slow RAM ("Trapdoor", damals oft "fast" genannt)	512 KB, C'00000–C7FFFF
Fast RAM	keins
Kickstart	zur Laufzeit NICHT vorhanden (Launcher-ROM im \$F8xxx-Bereich)
Floppy	nicht vorhanden (kommt mit späterem Milestone)
Eingaben	Tastatur, Joystick funktionieren; Maus folgt später

3. Was die Demo erfüllen muss (die harten Bedingungen)

Der Entry muss ein klassischer **Takeover-Entry** sein — also der normale Hauptteil-Einstieg eines Trackmos:

1. Er **initialisiert die komplette Hardware selbst von Null weg** (DMACON, INTENA, Copper, Bitplanes, eigene Interrupt-Vektoren, ...). Beim Einsprung läuft nichts: kein Copper, kein Display, keine Interrupts — siehe Abschnitt 4.
2. Er setzt **seinen eigenen Stack** (oder du nennst uns einen SP-Wert).
3. Er **kehrt nie zurück** und ruft ab dem Entry **nie mehr** etwas im ROM-Bereich auf — kein exec, kein graphics, auch kein Zugriff auf ExecBase über Adresse \$4.
4. **Kein Disk-Zugriff nach dem Entry**. Alles muss zum Dump-Zeitpunkt im RAM sein (Single-Load). Multi-Part-Trackmos, die nachladen, gehen erst mit dem Floppy-Milestone.
5. Nur **OCS-Register, PAL**.

Dein Instinkt mit dem Dump-Zeitpunkt war genau richtig: **Gedumpt wird in dem Moment, wo dein Packer/Loader den Speicher fertig organisiert hat und BEVOR der Hauptteil Hardware-Register anfasst**. Die Hardware-Register müssen (und können) nicht im Dump sein — sie werden von unserer Umgebung auf Reset-Zustand gestellt, und dein Hauptteil setzt ja ohnehin alles selbst.

4. Was wir beim Entry garantieren (der Vertrag)

Was	Zustand beim Einsprung
CPU	68000, Supervisor-Modus, SR = \$2700 (alle Interrupts maskiert)
PC	deine Entry-Adresse
SP (SSP)	dein Wunschwert; Default \$00080000; am robustesten: dein Code setzt ihn selbst als Erstes
Alle anderen Register	\$00000000 (sag Bescheid, falls du etwas anderes brauchst)
Custom-Chips	Reset-Zustand: DMACON = 0, INTENA = 0, INTREQ = 0, alle DMA aus, kein Copper aktiv
CIAs	Reset-Zustand
OVL	aus — Chip-RAM ab \$000000 sichtbar, inklusive der Vektortabelle aus deinem Dump
Chip RAM	\$000000–\$07FFFF = exakt dein Abbild; nicht gelieferte Bereiche sind 0
Slow RAM	C'00000–C7FFFF = exakt dein Abbild; nicht gelieferte Bereiche sind 0
ROM-Bereich	unser Launcher — ein Sprung dorthin ist ein Absturz

5. Was du lieferst

Variante A — Volldump (der einfachste Weg, empfohlen für den ersten Versuch):

1. `chip.bin` — exakt **524.288 Bytes** = \$000000–\$07FFFF
2. `slow.bin` — exakt **524.288 Bytes** = C'00000–C7FFFF (nur falls die Demo Slow RAM nutzt; sonst weglassen)
3. Eine kurze formlose Notiz: **Entry-Adresse**, **SP** (Wert oder “setzt mein Code selbst”), und falls relevant: Steuerung, Besonderheiten.

Variante B — direkt aus deinem Build (oft sauberer, wenn deine Packer-/Linker-Map die Speicherbelegung sowieso kennt):

1. Die fertigen Binär-Segmente als Dateien
2. Pro Segment die Ladeadresse (“`part1.bin` nach \$400, `part2.bin` nach \$20000, `musik.bin` nach \$C00000, ...”)
3. Dieselbe kurze Notiz (Entry, SP, Besonderheiten)

Übergabe formlos: ZIP per Discord/Mail. Größen sind unkritisch (max. ~1 MB).

Checkliste vor dem Abschicken (einmal kurz durchgehen):

- ☐ Entry ist ein Takeover-Entry (Hardware-Init komplett selbst)
- ☐ Nach dem Entry kein Kickstart-/ROM-Zugriff mehr (auch nicht \$4)
- ☐ Nach dem Entry kein Disk-Zugriff (Single-Load, alles im RAM)
- ☐ Nur OCS-Register, PAL
- ☐ `chip.bin` ist exakt 524.288 Bytes (`slow.bin` ebenso, falls dabei)
- ☐ Entry-Adresse und SP stehen in der Notiz

6. Schritt für Schritt: der WinUAE-Dump (Variante A)

1. **WinUAE konfigurieren** (entspricht exakt unserem Core):

- Quickstart: A500, Kickstart 1.3
- CPU: 68000, “cycle exact”
- Chipset: OCS / “Original”
- RAM: Chip 512 KB, Slow 512 KB, Fast 0

2. **Demo bis zum Dump-Moment laufen lassen.** Am elegantesten: bau vor den finalen JMP in den Hauptteil eine Endlosschleife (`bra.s *`) ein — du hast den Loader ja in der Hand. Alternativ im Debugger einen Breakpoint auf die Entry-Adresse setzen.

3. **Debugger öffnen:** Shift+F12. Nützliche Kommandos: h (Hilfe), f <adresse> (Breakpoint), r (Register anzeigen), d <adresse> (Disassembly), g (weiterlaufen).

4. **Speicher sichern** (Zahlen sind hex):

```
S chip.bin 0 80000  
S slow.bin c00000 80000
```

Die Dateien landen im WinUAE-Verzeichnis.

5. **Entry und SP notieren:** das JMP-Ziel ist die Entry-Adresse; r zeigt dir den aktuellen Registerstand, falls du den SP übernehmen willst. Fertig.

Wichtig: Der Dump-Moment muss VOR dem ersten Hardware-Zugriff des Hauptteils liegen. Wenn zwischen "Speicher fertig" und dem JMP noch CPU-only-Code läuft (Register aufräumen o. Ä.), ist das egal — die Regel ist nur: ab dem Entry darf nichts erwartet werden außer RAM-Inhalt und den Werten aus deiner Notiz.

7. Schritt für Schritt: aus dem Build (Variante B)

Wenn dein Build-System die finale Speicherbelegung kennt, brauchst du gar keinen Dump: Exportiere die fertigen (entpackten) Segmente als Dateien und schreib die Ladeadressen und den Entry in die Notiz. Wir setzen das Abbild daraus zusammen — Ergebnis ist identisch zu Variante A, nur ohne Emulator-Schritt und mit kleineren Dateien.

8. Was danach passiert

Wir packen deine Lieferung in unser RamDump-Format (eine Datei pro Titel), der Core-Loader lädt sie über das OSM-Menü von SD-Karte, und beim ersten Test iterieren wir gemeinsam — **der erste Wurf muss nicht perfekt sein**. Wenn deine Demo läuft, können mit derselben Methode weitere Single-Load-Titel aufbereitet werden: bei Fremdtiteln über die klassische One-Filer-Technik (Takeover-Entry suchen), später eventuell per Konverter direkt aus WinUAE-Savestates (inklusive generiertem Hardware-Restore-Stub — dafür muss am Core nichts geändert werden, die Komplexität steckt in der Datei). Multi-Load-Titel folgen mit dem Floppy-Support.

9. FAQ

- **Was, wenn die Demo beim Entry doch etwas voraussetzt, das fehlt?** Dann sehen wir das beim ersten Test (schwarzes Bild / Absturz) und iterieren. Die Lieferung kostet dich einmal 15 Minuten, kaputtgehen kann nichts.
- **Interrupt-Vektoren?** Liegen bei \$0-\$3FF im Chip-RAM und sind Teil deines Dumps. Ob sie zum Dump-Zeitpunkt schon installiert sind oder dein Hauptteil sie installiert, ist beides okay.
- **Muss der Speicher außerhalb meiner Daten leer sein?** Unsere Umgebung nullt vorher alles, was du nicht lieferst. Beim Volldump (Variante A) ist die Frage gegenstandslos.
- **Tastatur/Joystick in der Demo?** Funktioniert — CIA-A-Scancodes und Joystick-Ports sind verdrahtet. Maus kommt später; falls die Demo Maus braucht, kurz in der Notiz erwähnen.
- **Warum kein Kickstart zur Laufzeit?** Der Loader ersetzt das Kick-ROM durch einen Mini-Launcher, der nur OVL ausschaltet, SP/PC setzt und springt. Deshalb die Bedingung "keine ROM-Aufrufe nach Entry".